

# NullPointerException

## 1. Aim

To demonstrate the handling of NullPointerException in Java using appropriate exception-handling techniques.

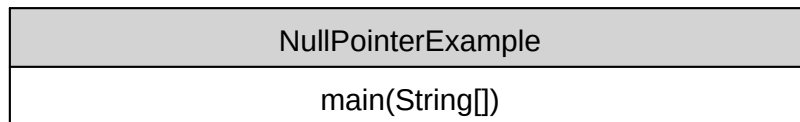
## 2. Description

Demonstrates handling a NullPointerException when a method is called on a null reference.

## 3. Algorithm

1. Start the program.
2. Set the String reference to null.
3. Print the name.
4. Call length() on the null reference, causing an exception.
5. Catch NullPointerException and display suitable messages.
6. End the program.

## 4. UML Diagram



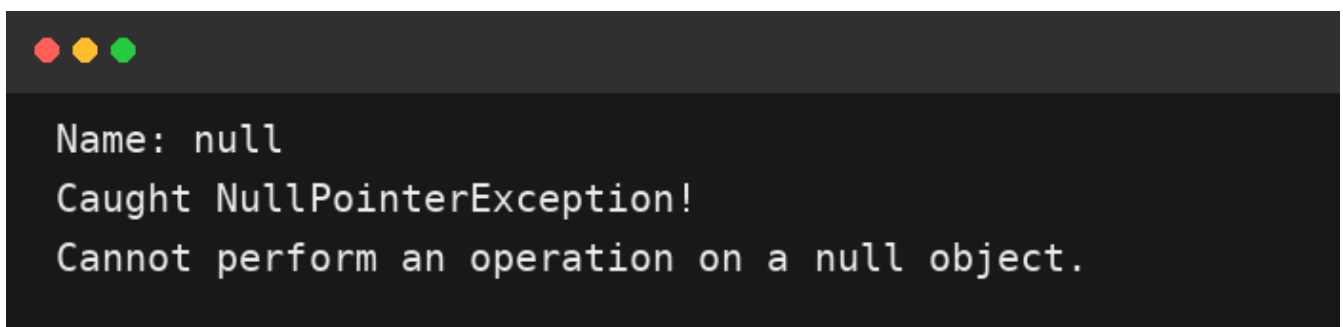
## 5. Source Code with Comments

```
public class NullPointerException {  
    public static void main(String[] args) {  
        try {  
            String name = null;  
  
            System.out.println("Name: " + name);  
  
            // Trying to call a method on a null reference  
            System.out.println("Length: " + name.length());  
  
        } catch (NullPointerException e) {  
            System.out.println("Caught NullPointerException!");  
            System.out.println("Cannot perform an operation on a null  
object.");  
        }  
    }  
}
```

## 6. Compilation and Execution Commands

```
javac NullPointerException.java  
java NullPointerException
```

## 7. Output Screenshot



```
Name: null  
Caught NullPointerException!  
Cannot perform an operation on a null object.
```

## 8. Result

The NullPointerException program was executed successfully and the required exception-handling behavior was demonstrated.

# ArrayIndexExample

## 1. Aim

To demonstrate the handling of ArrayIndexExample in Java using appropriate exception-handling techniques.

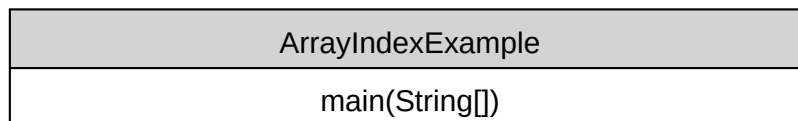
## 2. Description

Demonstrates handling an ArrayIndexOutOfBoundsException by intentionally accessing an invalid array index.

## 3. Algorithm

1. Start the program.
2. Read the array size and create the array.
3. Read elements into the array.
4. Read an invalid index.
5. Accessing the invalid index causes ArrayIndexOutOfBoundsException.
6. Catch the exception and display the error message.
7. Close the scanner and end the program.

## 4. UML Diagram



## 5. Source Code with Comments

```
import java.util.Scanner;

public class ArrayIndexExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            // 1. Ask the user for the array size dynamically
            System.out.print("Enter the size of the array: ");
            int size = scanner.nextInt();
            int[] dynamicArray = new int[size];

            // 2. Populate the array dynamically
            System.out.println("Enter " + size + " integers to fill the array:");
            for (int i = 0; i < size; i++) {
                System.out.print("Element at index " + i + ": ");
                dynamicArray[i] = scanner.nextInt();
            }

            // 3. Intentionally prompt for an invalid out-of-bounds index
            System.out.println("\n--- Triggering the Exception ---");
            System.out.print("Enter an out-of-bounds index to fetch
```

```

(e.g., " + size + " or higher): ");
int invalidIndex = scanner.nextInt();

// This line will throw ArrayIndexOutOfBoundsException if
the index is invalid
int value = dynamicArray[invalidIndex];
System.out.println("Value at index " + invalidIndex + " is:
" + value);

} catch (ArrayIndexOutOfBoundsException e) {
System.err.println("\n[ERROR] Caught
ArrayIndexOutOfBoundsException!");
System.err.println("Message: " + e.getMessage() + " is not a
valid index for this array.");
} finally {
scanner.close();
}
}
}
}

```

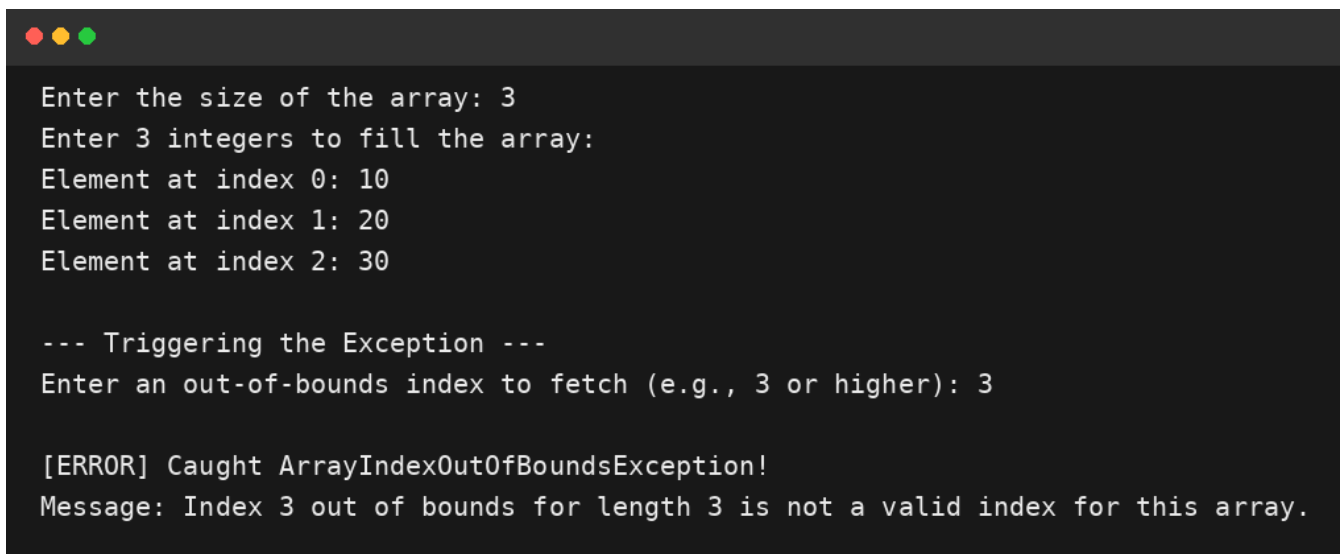
## 6. Compilation and Execution Commands

```

javac ArrayIndexExample.java
java ArrayIndexExample

```

## 7. Output Screenshot



```

Enter the size of the array: 3
Enter 3 integers to fill the array:
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30

--- Triggering the Exception ---
Enter an out-of-bounds index to fetch (e.g., 3 or higher): 3

[ERROR] Caught ArrayIndexOutOfBoundsException!
Message: Index 3 out of bounds for length 3 is not a valid index for this array.

```

## 8. Result

The ArrayIndexExample program was executed successfully and the required exception-handling behavior was demonstrated.

# NumberFormat

## 1. Aim

To demonstrate the handling of NumberFormat in Java using appropriate exception-handling techniques.

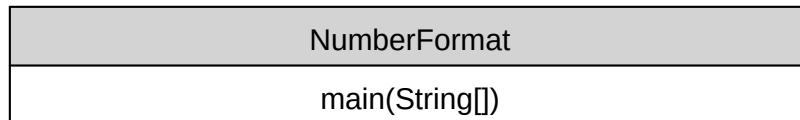
## 2. Description

Demonstrates handling a `NumberFormatException` when a non-numeric string is converted to an integer.

### 3. Algorithm

1. Start the program.
2. Store a non-numeric string.
3. Attempt to convert the string to an integer.
4. The conversion causes `NumberFormatException`.
5. Catch the exception and display a message.
6. End the program.

### 4. UML Diagram



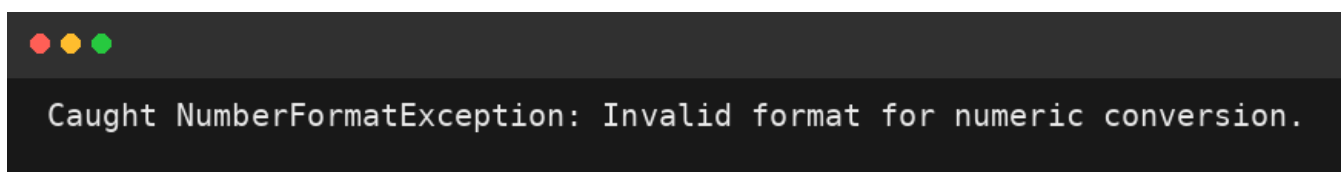
### 5. Source Code with Comments

```
public class NumberFormat {  
    public static void main(String[] args) {  
        try {  
            String invalidNumber = "abc";  
            int number = Integer.parseInt(invalidNumber); // Cannot  
            parse "abc" to int  
        } catch (NumberFormatException e) {  
            System.err.println("Caught NumberFormatException: Invalid  
            format for numeric conversion.");  
        }  
    }  
}
```

### 6. Compilation and Execution Commands

```
javac NumberFormat.java  
java NumberFormat
```

### 7. Output Screenshot



### 8. Result

The `NumberFormat` program was executed successfully and the required exception-handling behavior was demonstrated.

## StringIndex

### 1. Aim

To demonstrate the handling of `StringIndex` in Java using appropriate exception-handling techniques.

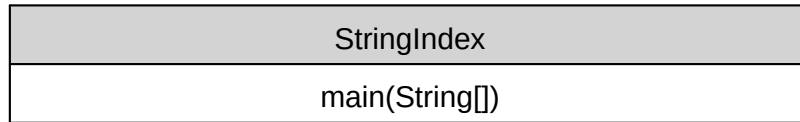
### 2. Description

Demonstrates handling a `StringIndexOutOfBoundsException` when an invalid string index is accessed.

### 3. Algorithm

1. Start the program.
2. Create the string "Hello".
3. Attempt to access character at index 10.
4. The index is outside the valid range, so an exception occurs.
5. Catch the exception and display a message.
6. End the program.

### 4. UML Diagram



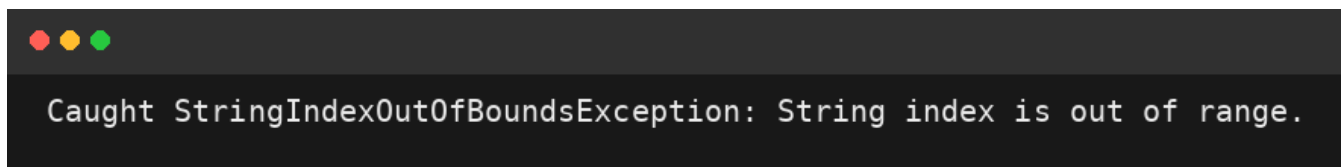
### 5. Source Code with Comments

```
public class StringIndex {  
    public static void main(String[] args) {  
        try {  
            String text = "Hello";  
            char ch = text.charAt(10); // String length is 5  
        } catch (StringIndexOutOfBoundsException e) {  
            System.err.println("Caught StringIndexOutOfBoundsException:  
String index is out of range.");  
        }  
    }  
}
```

### 6. Compilation and Execution Commands

```
javac StringIndex.java  
java StringIndex
```

### 7. Output Screenshot



### 8. Result

The StringIndex program was executed successfully and the required exception-handling behavior was demonstrated.

## IllegalArgument

### 1. Aim

To demonstrate the handling of IllegalArgument in Java using appropriate exception-handling techniques.

### 2. Description

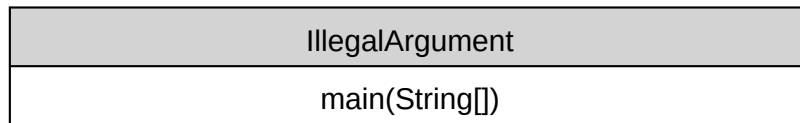
Demonstrates validation using IllegalArgumentException when a negative age is supplied.

### 3. Algorithm

1. Start the program.
2. Call setAge() with -5.

3. Check whether the age is negative.
4. Throw `IllegalArgumentException` if it is negative.
5. Catch the exception and display its message.
6. End the program.

## 4. UML Diagram



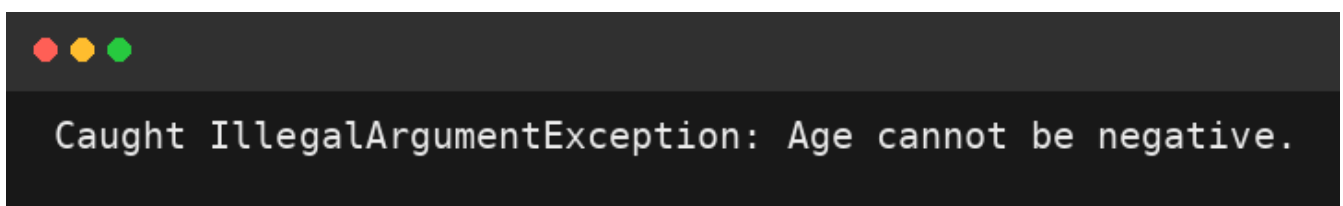
## 5. Source Code with Comments

```
public class IllegalArgumentException {  
    public static void setAge(int age) {  
        if (age < 0) {  
            throw new IllegalArgumentException("Age cannot be  
negative.");  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            setAge(-5);  
        } catch (IllegalArgumentException e) {  
            System.err.println("Caught IllegalArgumentException: " +  
e.getMessage());  
        }  
    }  
}
```

## 6. Compilation and Execution Commands

```
javac IllegalArgumentException.java  
java IllegalArgumentException
```

## 7. Output Screenshot



## 8. Result

The `IllegalArgumentException` program was executed successfully and the required exception-handling behavior was demonstrated.

# ClassCastException

## 1. Aim

To demonstrate the handling of `ClassCastException` in Java using appropriate exception-handling techniques.

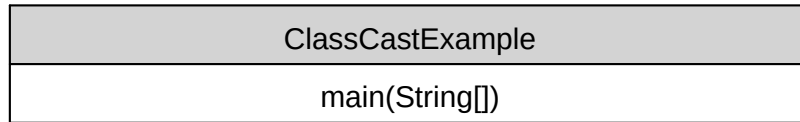
## 2. Description

Demonstrates handling a `ClassCastException` caused by an incompatible type cast.

### 3. Algorithm

1. Start the program.
2. Store a String object in an Object reference.
3. Attempt to cast the object to Integer.
4. The incompatible cast causes ClassCastException.
5. Catch the exception and display a message.
6. End the program.

### 4. UML Diagram



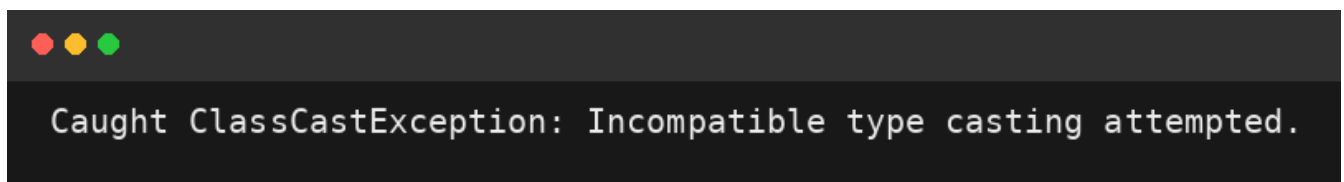
### 5. Source Code with Comments

```
public class ClassCastException {  
    public static void main(String[] args) {  
        try {  
            Object obj = "This is a string";  
            Integer num = (Integer) obj; // Cannot cast String to  
            Integer  
        } catch (ClassCastException e) {  
            System.err.println("Caught ClassCastException: Incompatible  
            type casting attempted.");  
        }  
    }  
}
```

### 6. Compilation and Execution Commands

```
javac ClassCastException.java  
java ClassCastException
```

### 7. Output Screenshot



### 8. Result

The ClassCastException program was executed successfully and the required exception-handling behavior was demonstrated.

## FileNotFoundException

### 1. Aim

To demonstrate the handling of FileNotFoundException in Java using appropriate exception-handling techniques.

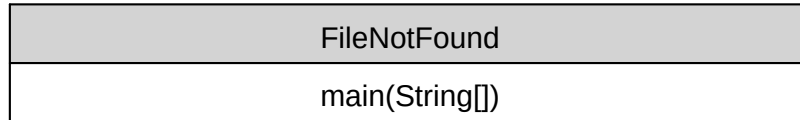
### 2. Description

Demonstrates handling a FileNotFoundException when the requested file cannot be located.

### 3. Algorithm

1. Start the program.
2. Create a File object for the required file.
3. Try to create a FileReader.
4. If the file does not exist, FileNotFoundException occurs.
5. Catch the exception and display a message.
6. End the program.

## 4. UML Diagram



## 5. Source Code with Comments

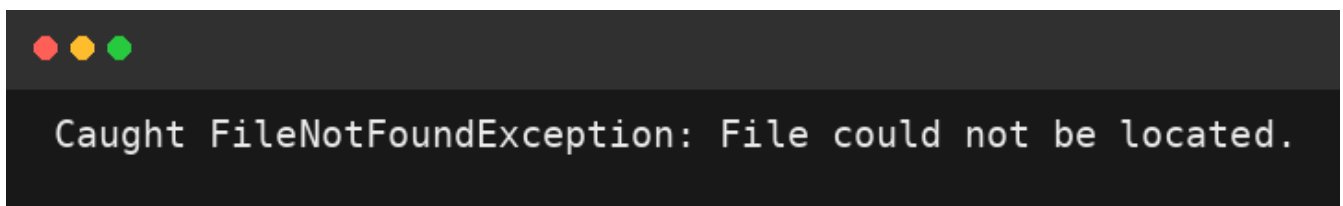
```
import java.io.File;
import java.io.FileReader;
import java.io.FileNotFoundException;

public class FileNotFoundException {
    public static void main(String[] args) {
        try {
            File file = new File("missing_file.txt");
            FileReader fr = new FileReader(file);
        } catch (FileNotFoundException e) {
            System.err.println("Caught FileNotFoundException: File could
            not be located.");
        }
    }
}
```

## 6. Compilation and Execution Commands

```
javac FileNotFoundException.java
java FileNotFoundException
```

## 7. Output Screenshot



## 8. Result

The FileNotFoundException program was executed successfully and the required exception-handling behavior was demonstrated.

# IOException

## 1. Aim

To demonstrate the handling of IOException in Java using appropriate exception-handling techniques.

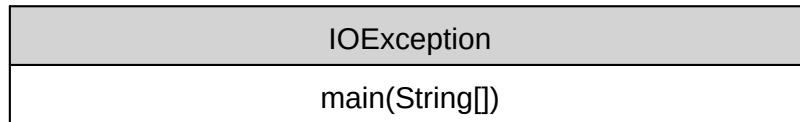
## 2. Description

Demonstrates handling an IOException during file input/output by reading from a closed FileReader.

## 3. Algorithm

1. Start the program.
2. Open test.txt using FileReader.
3. Close the reader.
4. Attempt to read from the closed reader.
5. The operation causes an IOException.
6. Catch the exception and display a message.
7. End the program.

## 4. UML Diagram



## 5. Source Code with Comments

```
import java.io.FileReader;
import java.io.IOException;

public class IOException {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("test.txt");
            reader.close();
            reader.read(); // Reading from a closed stream throws
                           // IOException
        } catch (IOException e) {
            System.err.println("Caught IOException: An I/O error
            occurred.");
        }
    }
}
```

## 6. Compilation and Execution Commands

```
javac IOException.java
java IOException
```

## 7. Output Screenshot



## 8. Result

The IOException program was executed successfully and the required exception-handling behavior was demonstrated.

# CustomException

## 1. Aim

To demonstrate the handling of CustomException in Java using appropriate exception-handling techniques.

## 2. Description

Demonstrates creation and handling of a custom checked exception for voting eligibility.

### 3. Algorithm

1. Start the program.
2. Define InvalidAgeException by extending Exception.
3. Call checkVotingEligibility(16).
4. If age is below 18, throw InvalidAgeException.
5. Catch the custom exception.
6. Display the exception message and end the program.

### 4. UML Diagram

| InvalidAgeException         | CustomException             |
|-----------------------------|-----------------------------|
| extends Exception           | checkVotingEligibility(int) |
| InvalidAgeException(String) | main(String[])              |

### 5. Source Code with Comments

```
class InvalidAgeException extends Exception {
public InvalidAgeException(String message) {
super(message);
}
}

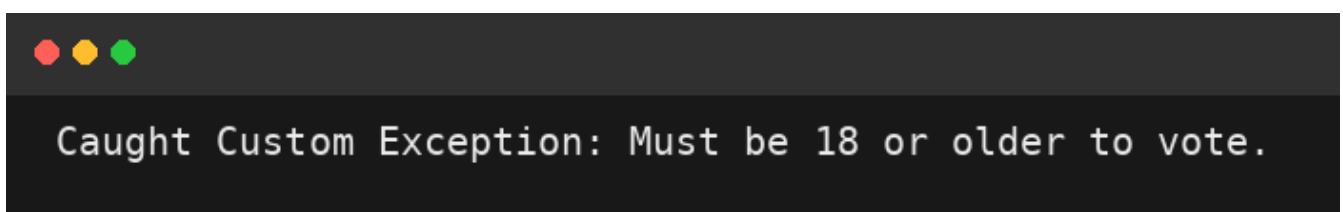
public class CustomException {
public static void checkVotingEligibility(int age) throws
InvalidAgeException {
if (age < 18) {
throw new InvalidAgeException("Must be 18 or older to
vote.");
}
}

public static void main(String[] args) {
try {
checkVotingEligibility(16);
} catch (InvalidAgeException e) {
System.err.println("Caught Custom Exception: " +
e.getMessage());
}
}
}
```

### 6. Compilation and Execution Commands

```
javac CustomException.java
java CustomException
```

### 7. Output Screenshot



### 8. Result

The CustomException program was executed successfully and the required exception-handling behavior was demonstrated.